```
> restart:

  VerifyConjecture118 := proc()

      local SturmBound, LastN, MaxDegree, H,
            periodic, numerator, euler,
            r, k, j, n, base, step, sign, residue,
            g1, g2, x,
            F, A, invF, two, precision, nextPrecision,
            modulusPoly, Fcut, temp, check, C,
            value, failures, startTime;

      startTime := time();

      # ------------------------------------------------------------
      # 1. Verification range supplied by the modular-form proof.
      # ------------------------------------------------------------

      SturmBound := iquo(26*839808, 12);
      LastN      := iquo(SturmBound - 48 - 20, 24);
      MaxDegree  := 24*LastN + 20;
      H          := iquo(MaxDegree, 2);

      if SturmBound <> 1819584
         or LastN <> 75813
         or MaxDegree <> 1819532 then
          error "Incorrect verification bound";
      end if;

      printf("Building the numerator modulo 3...\n");

      # ------------------------------------------------------------
      # 2. Periodic coefficients of
      #
      # (1+y)/(1-y)^15
      #   = (1+y)(1-y^3)(1-y^9)/(1-y^27)   (mod 3).
      # ------------------------------------------------------------

      periodic := Array(
          0..26, fill=0, datatype=integer[4]
      );

      for j in [0, 1, 12, 13] do
          periodic[j] := 1;
      end do;
```

```
for j in [3, 4, 9, 10] do
    periodic[j] := 2;
end do;

# ------------------------------------------------------------
# 3. Even-power part of the numerator of G_8(q).
#
# The exponent 3*r^2+17*r+8 is always even.
# After replacing q^(2*j) by x^j, the coefficients of
# the rational factor are periodic[2*k mod 27].
# ------------------------------------------------------------

numerator := Array(
    0..H, fill=0, datatype=integer[4]
);

r := 0;

while 3*r^2 + 17*r + 8 <= MaxDegree do

    base := iquo(3*r^2 + 17*r + 8, 2);
    step := 2*r + 1;
    sign := (-1)^r;

    residue := 0;

    for j from base by step to H do

        numerator[j] :=
            modp(
                numerator[j] + sign*periodic[residue],
                3
            );

        residue := irem(residue + 2, 27);

    end do;

    r := r + 1;

end do;

# ------------------------------------------------------------
# 4. Euler's pentagonal expansion of (x;x)_infinity.
#
# F(x) = 1 + sum_{k>=1} (-1)^k *
```

```
#           (x^(k*(3*k-1)/2) + x^(k*(3*k+1)/2)).
# ------------------------------------------------------------

euler := Array(
    0..H, fill=0, datatype=integer[4]
);

euler[0] := 1;
k := 1;

while iquo(k*(3*k-1), 2) <= H do

    g1 := iquo(k*(3*k-1), 2);
    g2 := iquo(k*(3*k+1), 2);

    sign := modp((-1)^k, 3);

    euler[g1] := sign;

    if g2 <= H then
        euler[g2] := sign;
    end if;

    k := k + 1;

end do;

F := modp1(
    ConvertIn([seq(euler[j], j=0..H)], x),
    3
);

A := modp1(
    ConvertIn([seq(numerator[j], j=0..H)], x),
    3
);

# ------------------------------------------------------------
# 5. Compute 1/F modulo x^(H+1) by Newton iteration:
#
# invF_new = invF*(2-F*invF).
#
# Every polynomial operation takes place modulo 3.
# ------------------------------------------------------------

printf("Computing the inverse Euler series modulo 3...\n");
```

```
invF := modp1(One(x), 3);
two  := modp1(Constant(2, x), 3);

precision := 1;

while precision < H+1 do

    nextPrecision := min(2*precision, H+1);

    modulusPoly := modp1(
        ConvertIn(x^nextPrecision, x),
        3
    );

    Fcut := modp1(
        Rem(F, modulusPoly),
        3
    );

    temp := modp1(
        Rem(Multiply(Fcut, invF), modulusPoly),
        3
    );

    temp := modp1(
        Subtract(two, temp),
        3
    );

    invF := modp1(
        Rem(Multiply(invF, temp), modulusPoly),
        3
    );

    precision := nextPrecision;

end do;

# Check the entire computed inverse series.

check := modp1(
    Rem(Multiply(F, invF), modulusPoly),
    3
);
```

```
    if not modp1(IsOne(check), 3) then
        error "The inverse-series check failed";
    end if;

    printf("Inverse-series check: PASS\n");

    # ------------------------------------------------------
    # 6. C(x) = sum D_8^0(2*j)*x^j modulo 3,
    #    through degree H.
    # ------------------------------------------------------

    printf("Multiplying the series and checking
coefficients...\n");

    C := modp1(
        Rem(Multiply(A, invF), modulusPoly),
        3
    );

    failures := 0;

    for n from 0 to LastN do

        value := modp1(
            Coeff(C, 12*n+10),
            3
        );

        if value <> 0 then

            if failures < 10 then
                printf(
                    "Failure: n=%d, argument=%d, residue=%d\n",
                    n, 24*n+20, value
                );
            end if;

            failures := failures + 1;

        end if;

    end do;

    # ------------------------------------------------------
    # 7. Report.
    # ------------------------------------------------------
```

```
        printf("\nSturm bound = %d\n", SturmBound);
        printf("Checked n = 0..%d\n", LastN);
        printf("Number of cases = %d\n", LastN+1);
        printf("Maximum argument = %d\n", MaxDegree);
        printf("Failures = %d\n", failures);

        if failures <> 0 then
            error "Finite verification failed";
        end if;

        printf("Finite verification: PASS\n");
        printf("CPU time = %.2f seconds\n", time()-startTime);

        return true;

    end proc:

    VerifyConjecture118():
Building the numerator modulo 3...
Computing the inverse Euler series modulo 3...
Inverse-series check: PASS
Multiplying the series and checking coefficients...

Sturm bound = 1819584
Checked n = 0..75813
Number of cases = 75814
Maximum argument = 1819532
Failures = 0
Finite verification: PASS
CPU time = 4677.75 seconds
>
```